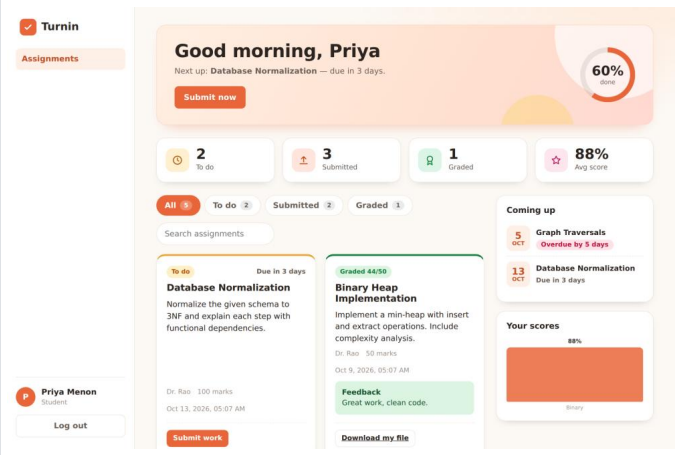
Turnin

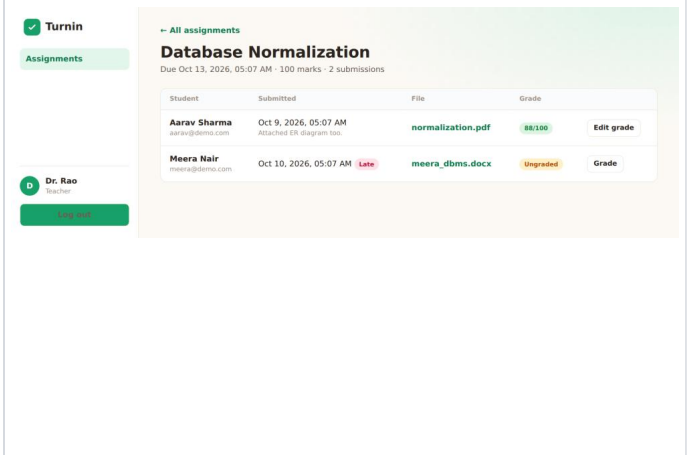
Assignment Submission Portal

Teachers publish assignments. Students submit files before the deadline.
Teachers grade with feedback. All in one secure place.

Node.js | Express | SQLite | JWT + bcrypt | BWT Hackathon



Student dashboard



Teacher: submissions and grading

About the Project

Teachers and students often exchange assignments through WhatsApp, e-mail or paper. This causes **lost files, missed deadlines, unclear submission status and delayed marks**.

Turnin is a secure backend web application that fixes this. Teachers publish assignments with a deadline and maximum marks, students upload their work before the deadline, and teachers download the files and grade them with feedback.

Built for the Backend Web Technologies (BWT) Hackathon, Department of Data Science, Malla Reddy Technical Campus.

Features

For teachers

- Create, edit and delete assignments (title, description, deadline, max marks)
- View all submissions with a Late tag
- Download submitted files
- Grade 0 to max marks with feedback

For students

- See status: not submitted, submitted, late, overdue or graded
- One file per assignment; resubmit until graded
- Dashboard with progress ring, counts, filters and search
- See marks and feedback

For everyone

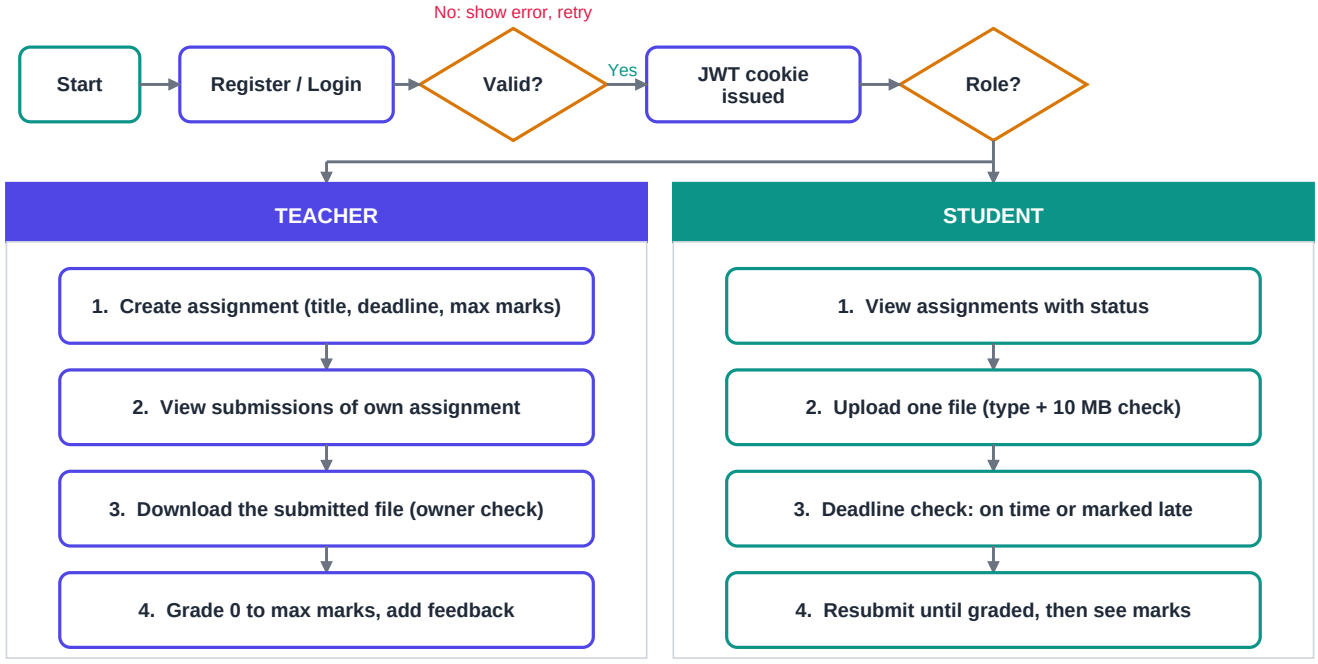
- Secure registration and login
- Role-based dashboards
- Automatic late detection against the deadline
- Each role sees only its own features

Tech Stack

Technology	Purpose	Why it is used
Node.js	Runtime	JavaScript on the server, with non-blocking I/O that suits file uploads.
Express	Web framework	Simple routing and middleware for the REST API and role checks.
SQLite (better-sqlite3)	Database	Zero-setup file database with relations and constraints.
JWT (httpOnly cookie)	Sessions	Stateless login. The cookie keeps the token away from page scripts.
bcrypt	Password hashing	Only hashes are stored, never real passwords.
multer	File uploads	Size limit, type whitelist and random file names.
helmet + rate limiting	Security	Safe HTTP headers and protection against brute-force logins.
HTML, CSS, vanilla JS	Frontend	Lightweight single-page UI with no framework needed.

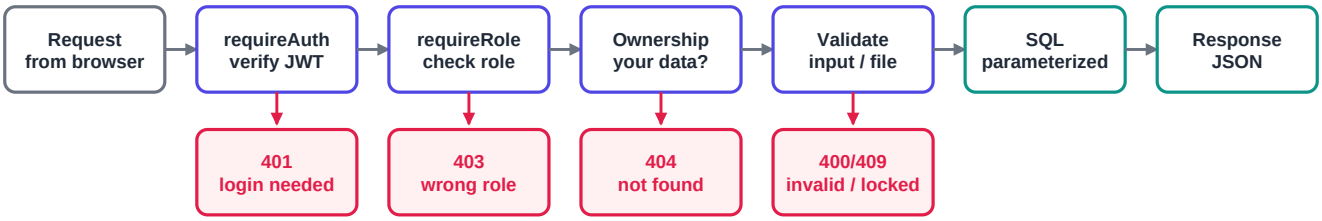
How It Works

User flow



Request flow

Every API call passes through the same checks. A failed check stops the request with an error.



Getting Started

Prerequisites

- Node.js (LTS version recommended)
- npm (comes with Node.js)

Installation

```
# 1. Clone the repository
git clone https://github.com/<your-username>/turnin.git
cd turnin

# 2. Install dependencies
npm install

# 3. Create a .env file (see below), then start the app
npm start
```

Environment variables (.env)

```
JWT_SECRET=replace_with_a_long_random_string
TEACHER_CODE=replace_with_a_secret_teacher_code
```

Variable	Description
JWT_SECRET	Secret used to sign login tokens. Keep it private.
TEACHER_CODE	Access code needed to register as a teacher. Students do not need it.

First steps

1. Register a **teacher** account using your TEACHER_CODE.
2. Register a **student** account.
3. As the teacher, create an assignment.
4. As the student, submit a file, then log back in as the teacher to grade it.

API Reference

Method	Route	Access
POST	`/api/auth/register, /login, /logout`	Public
GET	`/api/auth/me`	Logged in
GET	`/api/assignments`	Logged in (role-specific data)
POST / PUT / DELETE	`/api/assignments[/:id]`	Teacher (owner)
GET	`/api/assignments/:id/submissions`	Teacher (owner)
POST	`/api/assignments/:id/submit`	Student
GET	`/api/submissions/:id/file`	Owner student / owning teacher
PUT	`/api/submissions/:id/grade`	Owning teacher

Example: grade a submission

```
PUT /api/submissions/12/grade
Content-Type: application/json

{ "marks": 88, "feedback": "Great work, clean code." }
```

Marks must be a whole number from 0 to the assignment's maximum marks.

Database Design

users
id PK
name
email
password_hash
role (student/teacher)

assignments
id PK
teacher_id FK
title
description
deadline
max_marks

submissions
id PK
assignment_id FK
student_id FK
file_path
original_name
note
submitted_at
is_late
marks
feedback
graded_at

Relations: users (1) to (many) assignments via teacher_id; users (1) to (many) submissions via student_id; assignments (1) to (many) submissions via assignment_id.

UNIQUE (assignment_id, student_id) allows only one submission per student per assignment, and **ON DELETE CASCADE** keeps the data consistent.

Submissions table (SQL)

```
CREATE TABLE IF NOT EXISTS submissions (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  assignment_id INTEGER NOT NULL REFERENCES assignments (id) ON DELETE CASCADE,
  student_id    INTEGER NOT NULL REFERENCES users (id) ON DELETE CASCADE,
  file_path     TEXT NOT NULL,
  original_name TEXT NOT NULL,
  note          TEXT,
  submitted_at  TEXT NOT NULL,
  is_late       INTEGER NOT NULL DEFAULT 0,
  marks         INTEGER,
  feedback      TEXT,
  graded_at     TEXT,
  UNIQUE (assignment_id, student_id)
);
```

Security

- **Hashed passwords** using bcrypt.
- **JWT sessions** in an httpOnly cookie (sameSite: lax, 7-day expiry).
- **Role-based access** with requireAuth and requireRole middleware.
- **Ownership checks:** teachers reach only their own assignments; a file is served only to its student or the owning teacher.
- **Upload protection:** file-type whitelist, 10 MB limit and random file names.

- **Input validation** on every route, including the marks range.
- **helmet** security headers and **login rate limiting**.
- **Parameterized SQL queries** against SQL injection.

Allowed upload types: .pdf .doc .docx .txt .zip .ppt .pptx .png .jpg .jpeg .py .js .java .cpp .c

Project Structure

```
turnin/
|-- server.js            # App entry point
|-- db.js                # SQLite connection and table creation
|-- middleware.js        # requireAuth and requireRole
|-- routes/
|   |-- auth.js          # Register, login, logout, me
|   |-- assignments.js   # Assignments CRUD, upload and submit
|   `-- submissions.js   # File download and grading
|-- public/              # Frontend (HTML, CSS, vanilla JS)
|-- uploads/             # Uploaded files (random names)
`-- .env                 # JWT_SECRET, TEACHER_CODE (not committed)
```

Test Cases Handled

- Wrong password
- Duplicate e-mail
- Invalid teacher code
- Disallowed file type
- File larger than 10 MB
- Marks out of range
- Resubmitting after grading (locked)
- Opening another user's file

Future Scope

Deadline reminders by e-mail | **Multiple file uploads** | **Course-wise grouping** | **Grade export to CSV**

Team

Team 4, II Year B.Tech, Department of Data Science, Malla Reddy Technical Campus

S.No	Name	Roll No.	Role
1	A. Sai Charitha	25MVCSDR0005	Team Lead
2	G. Sathvika	25MVCSDR0020	
3	G. Ganesh	25MVCSDR0018	
4	D. Sai Kumar	25MVCSDR0014	

Faculty Mentor: Mrs. V. Nikitha Reddy

References

[1] Backend Web Technologies (BWT) class notes, Department of Data Science, Malla Reddy Technical Campus
[2] Express.js documentation, https://expressjs.com
[3] SQLite documentation, https://www.sqlite.org/docs.html
[4] MDN Web Docs (HTTP, REST and JavaScript), https://developer.mozilla.org
[5] JSON Web Token introduction, https://jwt.io/introduction